

Data Structures for Mapping, Memory, and Navigation in the Olympic Park

Group 5

March 24, 2026

Contents

1	Data collection and processing for mapping	2
1.1	Choosing of points	2
1.2	Justification of points	3
1.3	Justification of mapping decisions	3
1.4	Occupancy grid	3
2	Algorithms implementation	4
2.1	Linked List	5
2.2	KD-tree (Closest Landmark Query)	5
2.3	Graph Representation	5
2.4	Breadth-First Search (BFS)	5
2.5	Dijkstra's Algorithm (Array Implementation)	5
2.6	Dijkstra's Algorithm (Heap)	6
2.7	Scenario-based Discussion	6
2.7.1	Scenario 1: Mandatory Intermediate Location	6
2.7.2	Scenario 2: Full Tourist Guide Mode	6
2.7.3	Scenario 3: Repeated Route Queries	6
3	Human-Robot Interaction	6
4	Algorithms of numerical methods	7
4.1	Data Description	7
4.2	Methodology	7
4.2.1	Coordinate conversion	7
4.2.2	Velocity calculation	7
4.2.3	Filtering	8
4.2.4	Accelerometer integration	8
4.2.5	Motion pattern detection	8
4.3	Results	8
4.3.1	Velocity component plots	8
4.3.2	Speed comparison plot	9
4.3.3	Motion pattern detection plot	9
4.4	Discussion	9
4.4.1	Why GPS results reliable	9
4.4.2	Why filtering necessary	9
4.4.3	Why pattern detection method acceptable	10
4.5	Conclusion	10

1 Data collection and processing for mapping

1.1 Choosing of points



7 signal points

- S1 - London Stadium
51.537876486551404, -0.014981956825029601
- S2 - Riverside East (Stratford)
51.53877478430856, -0.012202774139335007
- S3 - Zip World London at ArcelorMittal Orbit
51.53856431625901, -0.012965712150750694
- S4 - UCL East One Pool Street
51.53836793515591, -0.009771259329826044
- S5 - West Ham United Stadium Store
51.537361411587135, -0.015310747753419536
- S6 - UCL East Marshgate
51.537403345082026, -0.011619213141930713
- S7 - Bobby Moore Academy (Secondary)
51.536450286907666, -0.013968621757320058

10 Key points

- K1 + Waiting area 1 - The area in front of the main entrance on the east side of S1
51.53782115888637, -0.014409032791812363
- K2 - The area between S1 and S5
51.537417493783494, -0.015578306395135043
- K3 - The T-shaped intersection between S5 and K1
51.53743728996587, -0.01465389100353117
- K4 - A southern area of S7
51.53580687612669, -0.013666239418904718
- K5 - A bridge on the southern side of S6
51.5357914205134, -0.012553635489149489

- K6 - A southeast area of S6
51.537131735946545, -0.010170437019962977
- K7 + Waiting area 2 - Iron Bridge
51.53753812280469, -0.01022625086775594
- K8 - The open space to the west of S4
51.53852776356518, -0.01031324013731113
- K9 - The open space between S2 and S6
51.53817058717565, -0.011950396751371875
- K10 - The open space between S2 and S3
51.538483305426816, -0.012422720414710731

1.2 Justification of points

K1, K7 (Waiting Areas):

They are located in spacious open areas that are suitable for robots to park without blocking pedestrians. At the same time, open areas can ensure the best visibility of the GPS satellite, reducing positioning drift.

K3, K9, K10:

These are the "nodes" of the path. Selecting them can simplify the topology structure, allowing the Dijkstra algorithm to plan the shortest and safest path in a complex building complex.

K5, K7:

The bridge serves as the sole passage. The installation of signal points at these locations is to ensure that the robot has a clear navigation target point when crossing complex terrain such as rivers.

1.3 Justification of mapping decisions

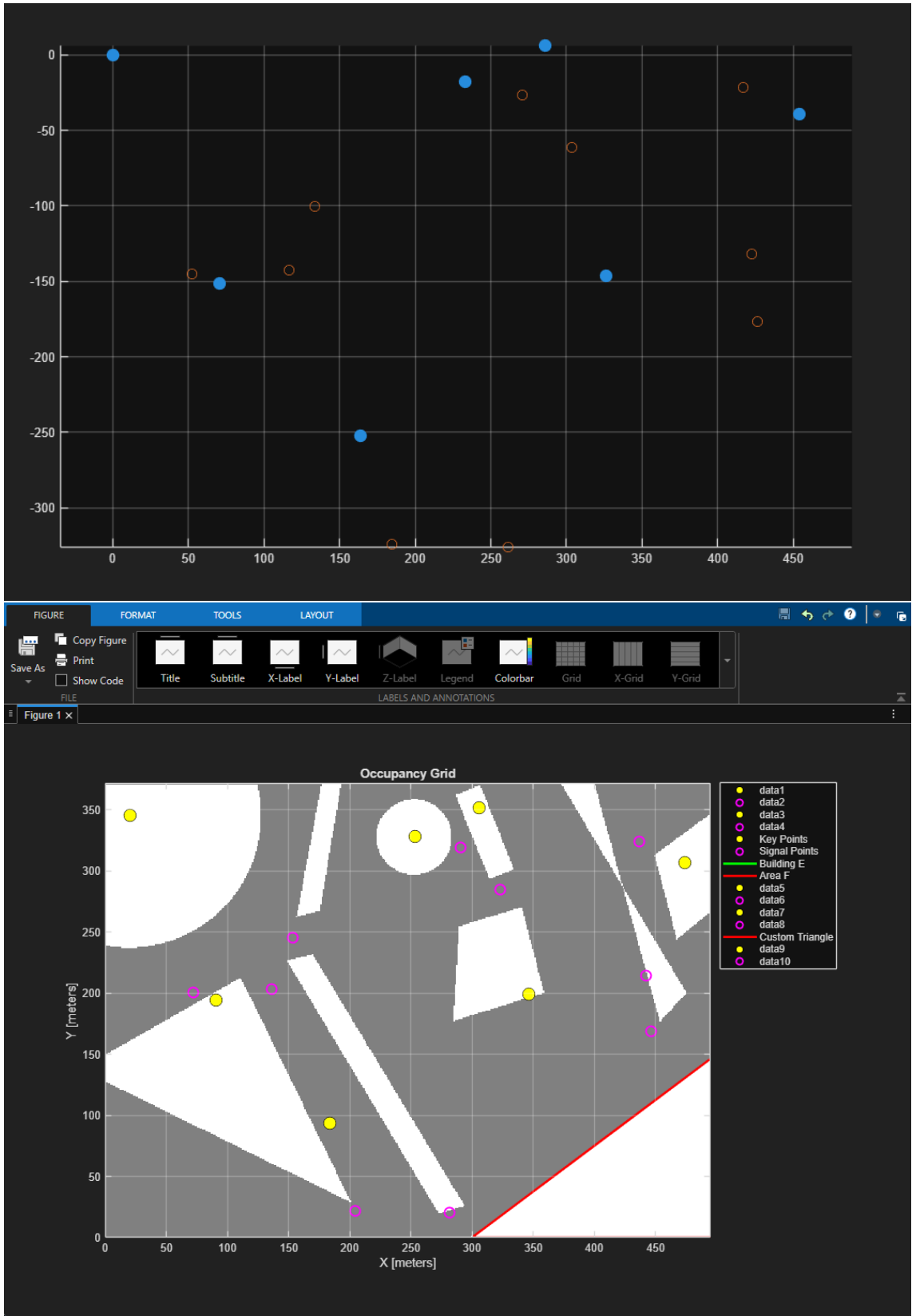
Instead of relying solely on the noisy GPS trajectories collected via MATLAB Mobile, I strategically employed Google Maps to extract precise geodetic coordinates for static obstacles. This decision was taken to mitigate GPS drift—often exceeding 5 meters near large structures like the London Stadium. By defining buildings as polygons and circles within the Occupancy Grid, the representation achieves higher fidelity and ensures safety for subsequent path-planning tasks.

Furthermore, the 17 points were distributed as a "Topological Skeleton." Signal points (S-points) were placed in open areas to maximize GPS signal quality and at critical choke points (e.g., the Iron Bridge) to guarantee navigable paths across the water features. The chosen resolution of 1m/cell balances the spatial complexity ($O(n \times m)$), while n is Width in Cells, m is Height in Cells) with the necessity of representing physical passages for the robot assistant.

1.4 Occupancy grid

The Matlab code for generating the grid is located in a file named "Task1-Grid".

Solid dots represent key points, hollow dots represent Signal points, and the white areas indicate obstacles (buildings or rivers).



2 Algorithms implementation

In this task, several data structures and graph search algorithms were implemented to support a robotic navigation system. We evaluate them in terms of efficiency (time complexity), memory usage, and suitability for different navigation scenarios.

2.1 Linked List

A linked list is used to store the key points and signal points in a flexible, dynamically extensible structure. Compared with array-based storage, linked lists support efficient insertion and deletion without reallocating contiguous memory.

Complexity: Linear lookup requires traversal:

$$T(n) = O(n).$$

Therefore, linked lists are suitable for small-scale storage but become inefficient for nearest-point queries as the number of nodes increases.

2.2 KD-tree (Closest Landmark Query)

A KD-tree is implemented to store the seven key landmarks (K1–K7). Unlike a linked list, a KD-tree organises points spatially by recursively partitioning the space along alternating axes. This structure supports efficient nearest neighbour (closest landmark) queries.

Complexity: Average nearest neighbour query cost is:

$$T(n) = O(\log n),$$

compared with linear search:

$$T(n) = O(n).$$

Although the difference may be small for only seven landmarks, the KD-tree becomes increasingly advantageous as the dataset grows. This makes it suitable for selecting the closest landmark in “tourist-guide” style behaviours.

2.3 Graph Representation

The map is represented as a weighted graph using an adjacency list. Nodes represent key points and signal points. Edges represent walkable connections, and edge weights represent geographical distances.

Space complexity: The adjacency list stores all vertices and only existing edges:

$$S = O(V + E),$$

which is memory-efficient for sparse navigation maps where each location connects to only a few others.

2.4 Breadth-First Search (BFS)

BFS is implemented to find a path with the minimum number of steps (edges) between two nodes. The advantages of BFS are simple and fast, however, minimum steps don’t mean shortest distance, so, it can’t guarantee shortest distance. Therefore, BFS is suitable for scenarios where all edges have equal cost or the objective is to minimize transitions rather than travel distance.

Complexity:

$$T = O(V + E).$$

Limitation: BFS ignores edge weights, therefore the minimum-step path does not necessarily correspond to the minimum-distance path. BFS is most appropriate when all edges have equal cost or when minimising transitions is more important than minimising physical distance.

2.5 Dijkstra’s Algorithm (Array Implementation)

The array-based Dijkstra implementation computes the shortest distance path by repeatedly selecting, among all unvisited nodes, the node with the smallest tentative distance. This selection is performed by scanning all nodes at each iteration.

Complexity:

$$T = O(V^2),$$

which can be slow for repeated queries, although it guarantees the optimal (minimum-distance) path and is straightforward to implement.

2.6 Dijkstra’s Algorithm (Heap)

Unlike array-based implementation to scan all nodes before choosing, the heap-based Dijkstra improves efficiency by using a priority queue to select the next node.

Complexity:

$$T = O((V + E) \log V).$$

The heap implementation typically uses slightly more memory due to the priority queue and is more complex to implement, but it is significantly faster for large graphs and for real-time or repeated navigation queries.

2.7 Scenario-based Discussion

2.7.1 Scenario 1: Mandatory Intermediate Location

In some navigation tasks, the robot must pass through a specific location before reaching its final destination.

For example, when travelling from K6 to K4, the robot may be required to pass through S9, resulting in the route:

$$K6 \rightarrow S9 \rightarrow K4$$

In this scenario, it is necessary not only to ensure that the intermediate point is visited, but also that the overall route remains as short as possible.

Dijkstra’s algorithm with a heap-based implementation is suitable for this task because it guarantees the shortest path while efficiently handling multi-stage routing.

Compared to the array-based implementation, the heap-based version is faster.

BFS can find a feasible path but does not guarantee the shortest distance.

2.7.2 Scenario 2: Full Tourist Guide Mode

In the tourist-guide mode, the robot is required to visit all key locations.

The KD-tree can efficiently select the nearest next landmark, while Dijkstra’s algorithm (heap implementation) computes the shortest route between consecutive locations.

By combining KD-tree for nearest-point selection and Dijkstra (heap) for path planning, both decision speed and routing accuracy can be improved.

2.7.3 Scenario 3: Repeated Route Queries

In situations where multiple route computations are required, such as when the robot needs to continuously navigate between different locations, path planning must be performed repeatedly.

For example, after reaching one destination, the robot may need to immediately compute a new route to the next location.

In this case, the heap-based Dijkstra implementation is more efficient than the array-based version, as it reduces the computational cost of repeated shortest-path calculations.

3 Human-Robot Interaction

1. Design Decision: The Necessity of GUI

- Reducing Cognitive Load: By selecting destinations (such as K1, K2) through dropdown menus, complex geographic coordinates are abstracted into easily understandable landmark names.
- Intuitive Interaction: The GUI provides a visual context of the environment. Real-time rendering of the path on the occupancyMap enables users to immediately understand why the robot is taking a detour, thereby enhancing users’ trust in the system.

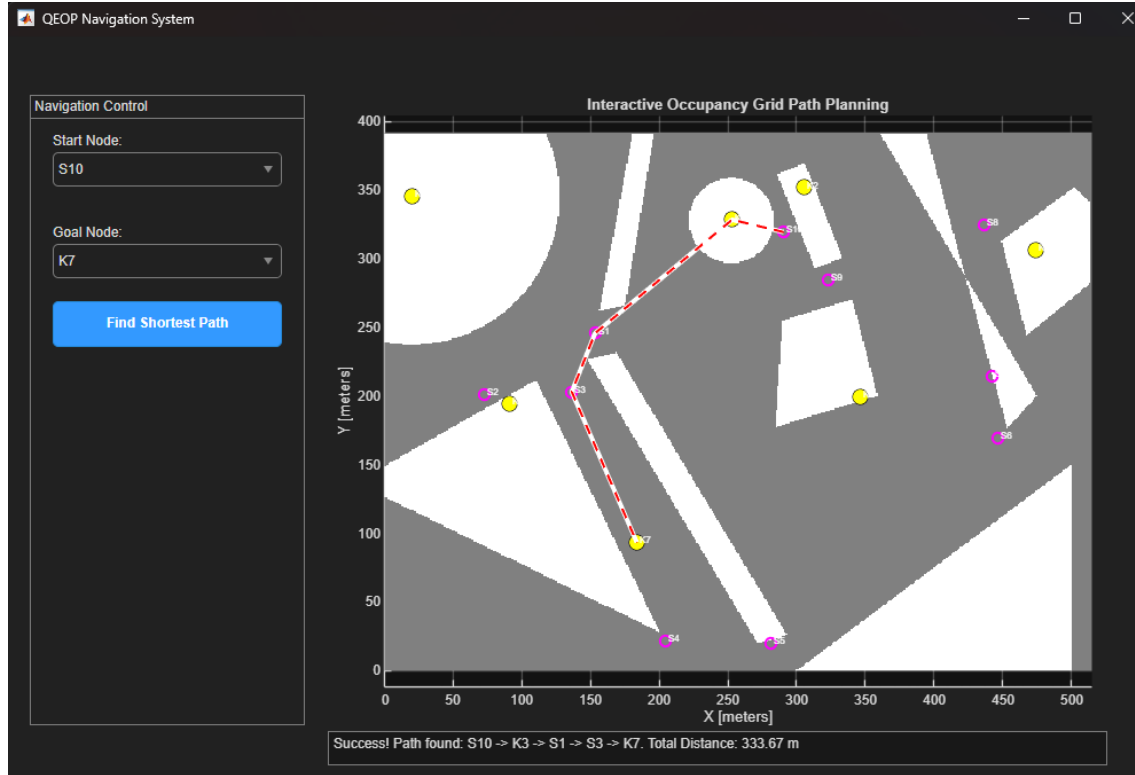
2. Functional Decision: Status Feedback

- System Transparency: The Status Bar displays real-time information such as "Planning in Progress" and "Arrived", which complies with the principle of "System State Visibility" in human-computer interaction.

- **Calculation Feedback:** The total distance calculated by the Dijkstra algorithm is directly displayed on the interface, providing users with decision-making references (for example: Users can judge whether to choose to walk based on the distance).

3. Algorithm Integration: Event-Driven Architecture

- **Real-time Response:** By using Nested Functions, the algorithm can respond to user click events immediately. This "on-demand computing" model saves more computing resources compared to pre-computing all paths.
- **Data Consistency:** The GUI view, map obstacle data, and path planning algorithm share the same translated coordinate system, ensuring "what you see is what you get" and avoiding navigation errors caused by positioning drift.



4 Algorithms of numerical methods

4.1 Data Description

The data in Part II were obtained from tests in MATLAB Mobile. Two types of sensor data were mainly used, namely acceleration and position. The acceleration data includes x, y, z with the unit m/s^2 . The position data includes latitude, longitude, speed, heading, altitude and horizontal accuracy. Both datasets laid the foundation for subsequent velocity estimation, filtering, comparative analysis and motion pattern recognition.

4.2 Methodology

4.2.1 Coordinate conversion

Since the data collected in the position data needs to be converted into a coordinate system first, the first GPS point is selected as the origin of this coordinate system, the latitude and longitude are converted into the x -axis and y -axis respectively, the altitude is converted into the z -axis, and then we can use numerical differentiation to analyze the trajectory data.

4.2.2 Velocity calculation

The velocity components in the x, y , and z directions were computed using finite differences with respect to time. If $x(t)$, $y(t)$, and $z(t)$ denote the local position coordinates, then the corresponding

velocity components were estimated as:

$$v_x = \frac{\Delta x}{\Delta t}, \quad v_y = \frac{\Delta y}{\Delta t}, \quad v_z = \frac{\Delta z}{\Delta t},$$

The total speed was then calculated as:

$$v = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

This provided a GPS-derived estimate of pedestrian motion.

4.2.3 Filtering

The original speed data shows some short-term fluctuations, which are caused by noise from GPS measurements. To reduce these fluctuations, I applied a moving average filter with the same window size to the speed data in the x, y, and z directions. The reason for using the same filter parameters in all directions is to ensure that the speed data in the x, y, and z directions can be compared together with consistent and reliable results. After filtering the speeds in all three directions, I used the processed speed data to calculate the overall smooth speed.

4.2.4 Accelerometer integration

Besides velocity estimation from GPS, we also used acceleration measurements from the mobile phone and estimated velocity through numerical integration. Before integration, we first corrected the acceleration signal to reduce the effects from gravity and low-frequency drift. Then we applied the cumulative trapezoidal integration method to perform time integration on the corrected acceleration signal. Finally, the velocity estimated from acceleration was compared with the velocity derived from GPS to evaluate the consistency between the two methods.

4.2.5 Motion pattern detection

We used the threshold method to distinguish different motion patterns. With the filtered total speed, we can judge whether a person is stationary, walking slowly or walking quickly. To determine whether a person turns, we detect it by observing the GPS heading angle. If this value changes significantly and the speed is higher than the stationary threshold, we determine this behavior as turning. Finally, we mark all obtained states onto the trajectory.

4.3 Results

4.3.1 Velocity component plots

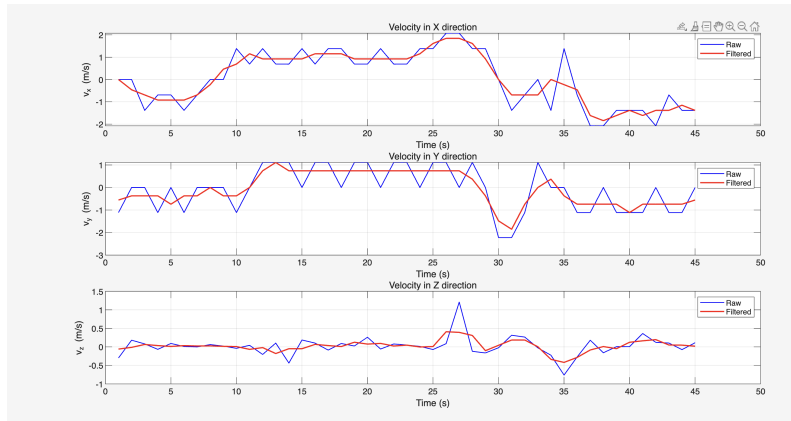
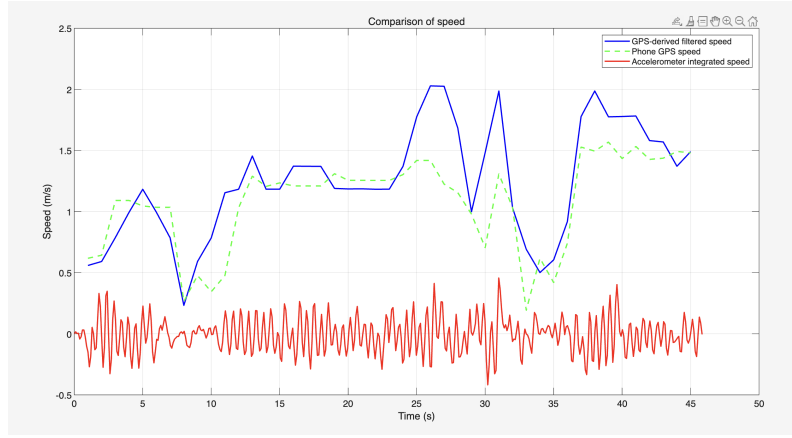


Figure 1 shows the raw and filtered velocities in the x, y, and z directions. The changes in the x and y directions are greater than those in the z direction, which shows that the movement during walking is mainly horizontal. In the figure, the velocity in the z direction is close to zero because the height does not change much, so this is reasonable. After filtering, the velocity curves become smoother without changing the trend of the motion.



4.3.2 Speed comparison plot

Figure 2 shows three different speed estimation results: the filtered GPS speed, the speed directly measured by the phone, and the speed obtained by integrating the measured acceleration data. It is observed that the overall trend of the filtered GPS speed and the phone GPS speed is very close, which indicates that the finite difference method can reasonably estimate the motion state. In contrast, even after data correction, the speed obtained from acceleration is still irregular and has poor stability.

4.3.3 Motion pattern detection plot

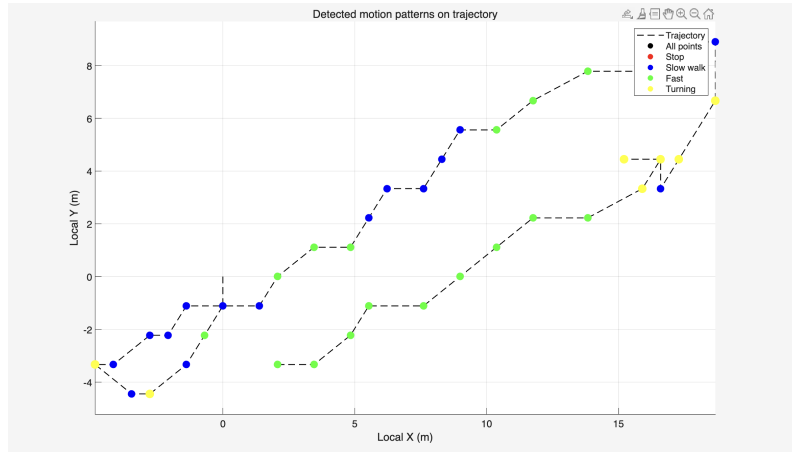


Figure 3 shows the detected motion patterns marked during walking. We found 1 stop, 18 slow walks, 18 fast walks and 8 turns. Most of the time we switched between slow walking and fast walking, and there were several turns during the process. Although only one stop was found, this is reasonable because this was a short-time walking experiment to obtain experimental data.

4.4 Discussion

4.4.1 Why GPS results reliable

In the experiment, we can conclude that the speed estimated by GPS is more reliable and accurate than the speed obtained from acceleration. GPS speed is derived from position changes, while acceleration speed is obtained indirectly through integration. During this process, the errors of acceleration accumulate over time.

4.4.2 Why filtering necessary

We need filtering because the original GPS speed has obvious fluctuations caused by measurement noise and limited sampling resolution. If we don't filter the data, the speed signal will be more complicated to understand and will have a bad effect on the later motion pattern recognition. The

moving average filter is simple and effective. It smooths the signal while keeping the main trend of the movement.

4.4.3 Why pattern detection method acceptable

The threshold-based method works well for our data because the experiment was short and the movement types were simple. This method can effectively distinguish different motion patterns, and combined with changes in the heading angle, we can more clearly identify the turning state.

4.5 Conclusion

In summary, Part II analyzes pedestrian motion data collected by MATLAB Mobile, using different sensors to collect information. First, the GPS position data is converted into a local coordinate system (X, Y, Z), and then the finite difference method is used to calculate the velocity. Filtering improves the interpretability of the velocity signal, and acceleration integration provides additional velocity estimation results for comparative analysis with GPS velocity estimation. The results show that velocity estimation based on GPS is more reliable than velocity estimation based on acceleration. Meanwhile, different motion states are identified and labeled: standing still, slow walking, fast moving, and turning.